

INTRODUCTION TO C PROGRAMMING :-

→ Programming language is a system of words, symbols and rules used to write instructions for computer to perform tasks.

Types Of Programming Languages :-

1 - Low - Level language :-

- machine language
- Assembly language
- fast but difficult for humans

2 - High - level languages :-

- C/C++/Java/Python etc
- Easy to read/write

- 'C' is considered as middle level because :-

- it support system-level operations
- Also offers high-level constructs

BASICS OF 'C' PROGRAMMING :-

- Developed By Dennis Ritchie in 1972
- ✓ Used in operating systems, embedded systems, compilers, game programming
- compiled language source code - machine language through compiler.

Features Of 'C' :-

- Simple and powerful
- Platform independent

Support pointers (powerful memory Access)

Fast execution

Modular (support functions)

Rich library.

STRUCTURE OF A 'C' PROGRAM :-

Every C program has -

- Documentation Section
- Link section (#include <stdio.h>)
- Definition Section
- Global Declaration Section
- Main() function
- Subprograms (function)

Ex:-

```
#include <stdio.h>
```

Header file

```
int main ()
```

standard input output

```
{
```

```
printf ("Hello C");
```

Main function

```
return 0;
```

printf() function is used to display

```
}
```

1. KEYWORDS :-

C has 32 key words:-

That are:-

auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, type, union, unsigned, void, volatile, while.

Identifiers :-

- Names given to variable, function, arrays, structures is called as identifiers.

- Rules:-

- Can't start with number

- No special character except underscore ('_')

- cannot use keywords

case-sensitive

- Meaningful names preferred.

Ex:-

roll-no, sum # valid

1sum, @ab, a@b # invalid

CHARACTER SET :-

The character that can be used for words, numbers, expressions is known as character set.

→ It is divided in 4 categories.

1. letters - A to Z and a to z
2. Digit - 0 to 9
3. special character - @, #, !, —, " ", ;
4. White-space - blank space, horizontal tab, carriage return, New line, form feed.

C token

- The smallest individual unit in a programme is called as token

- There are 6 types of token.

- 1- keyword, 2- identifiers, 3- constant, 4- string, 5- special symbol
- 6- operators

3. CONSTANT :-

→ It refers to the fixed value that don't change during execution of a program.

→ It is 2 types:-

1- Numeric - integer, real

2- Character - single char, string

BACK SLASH CHARACTER :-

- C support some back slash character constant that are used in output function.

CONSTANT

MEANING

\a

- Audible Alert

\b

- back space

\f

- form feed

\n

- New line

\r

- Carriage return

\t

- horizontal tab

\v

- Vertical tab

\0

- Null

4. Operator:-

- Operator is a symbol that is used to perform specific task.
Ex:- +, -, *, /, %.

5. Variable:-

Variables are memory locations given a name to store values.

Ex- `int a;` ^{variable} `float marks;`

Rules to Write a Variable:-

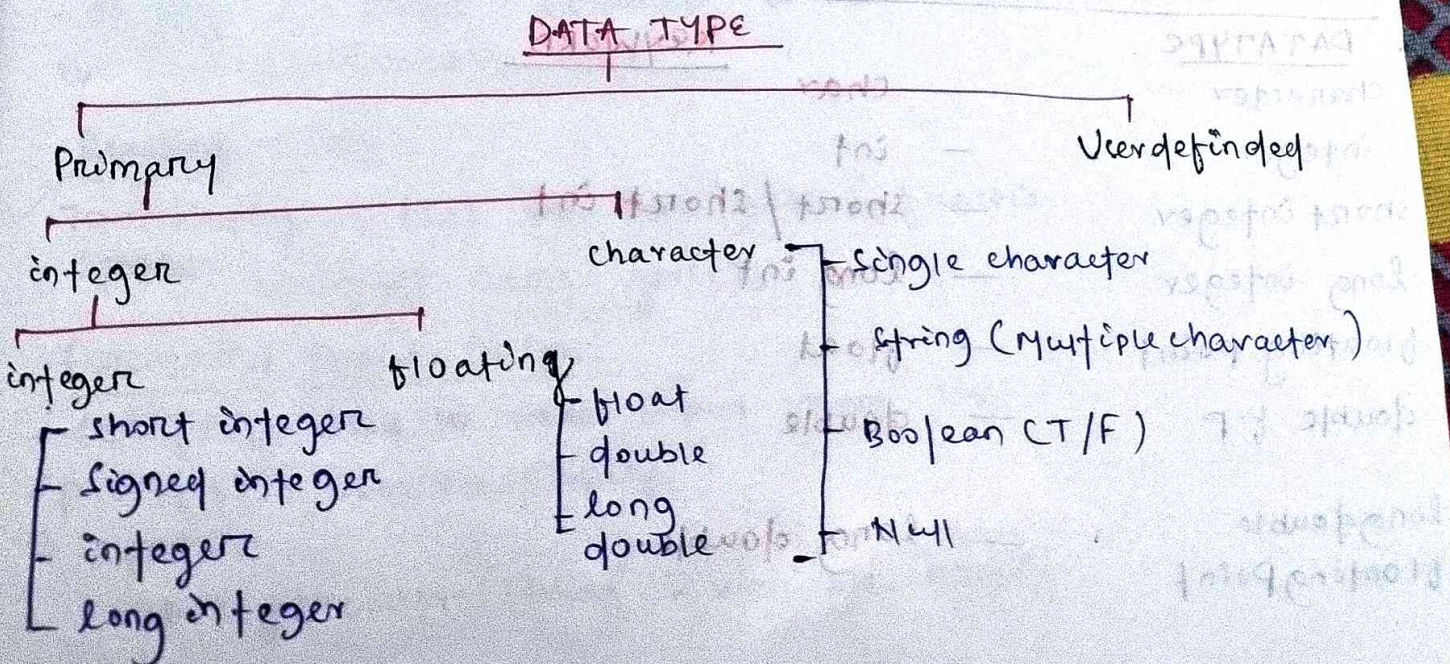
1. It means/variable name consist of letters, digits and underscore
2. Must begin with a letter. `int a1;` - valid `int 1a;` - invalid
or underscore
3. Case-sensitive
4. White space are not allowed
5. It should not be a keyword.

DATA TYPES:-

- It defines the type of data a variable can hold.
- It tells the compiler how much memory to allocate and what kind of operation can be performed.

There are 2 types:-

1. Primary data type
2. User define data type



Size And Range OF DATA TYPES :-

<u>Data Type</u>	<u>Size</u>	<u>RANGE</u>
Character (signed)	- 8 bit / 1 byte	
Unsigned char	- 1 byte	
short int	- 2 byte	
integer	- 4 byte	
long int	- 4 byte	
floating point	- 4 byte	
Double	- 8 byte	
long double	- 10 byte	

Declaration Variable :-

- It tells the compiler what the variable name is
- It specifies what type of value the variable will hold
- Syntax of variable declaration

<datatype> variable_name;

Ex:- int class-4;

↙ datatype ↘ variable

KEYWORDS OF DATATYPE :-

<u>DATATYPE</u>	<u>KEYWORD</u>
character	- char
integer	- int
short integer	- short / short int
long integer	- long int
floating point	- float
double f.p	- double
long double	- long double
floating point	

FORMATTED OUTPUT FUNCTION :-

Syntax:- `printf("control string", variable_name);`

DATA TYPE

CONTROL STRING

integer	_____	%d
char	_____	%c
str	_____	%s
long	_____	%ld
float	_____	%f
long double	_____	%L
short int	_____	%h
double	_____	%lf

Managing Input Output Operation :-

Read A single character :-

- To read a single character we used '`getchar()`'-function.
- ✓ To store that single character,

Syntax- `variable_name = getchar();`

Ex:-
`char ch;`
`ch = getchar();`

Display a single character :-

- To display the single character we use `putchar()`;
- It display single character to the console terminal

Ex:- `putchar(variable_name);`

Reading a line :-

- ✓ To reading a line we use '`getsc()`'-function.

Syntax:- `Variable_name = getsc();`

Formatted Input-Output

`scanf()`; - Used to take input from user.

- `printf()`; - Used to display,

→ Syntax:-

`scanf("control string", &variable_name);`

Operands :-

It is the data which operators operates.

Operators :-

Defn:- It is a symbol that ^{is used to} perform a specific operation.

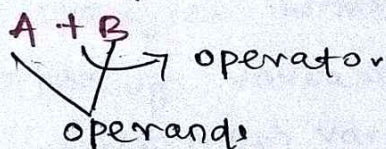
Types of operators :-

- 1- Arithmetic
- 2- Relational
- 3- Logical
- 4- Assignment
- 5- Increment decrement
- 6- Special
- 7- Bitwise

ARITHMETIC OPERATOR

+, -, /, *, %

→ These operators are known as binary operator



RELATIONAL OPERATORS

< - less than

<= - less than equal to

> - Greater than

>= - Greater than equal

== - Equal to

!= - Not Equal to

LOGICAL OPERATOR

&& - AND (into)

!! - OR (plus)

! - NOT

ASSIGNMENT OPERATOR

→ It means assign value to variable for expression.

= Equal to

+= Addition Equal to

- = Subtraction Equal to

* = multiply Equal to

/ = divided by Equal to

% = Modulus Equal to

Ex

$a += b$ means
 $\Rightarrow a = a + b$

INCREMENT, DECREMENT OPERATOR

→ INCREMENT - Increase the value by 1

Ex $a = 5$

$++a / a++ = 6$

- It is denoted as $++$.

- DECREMENT - Decrease value by 1

Ex $a = 5$

$a-- ; \# 4$

- It is denoted as $--$.

CONDITIONAL OPERATOR: ("?:")

→ It takes three operator
Syntax

$Exp1 ? Exp2 : Exp3$

→ first $Exp1$ evaluates if true then Expression 2

but $Exp1$ is false then Expression 3 evaluated.

Special Operator :-

C support some special character.

1 - comma operator (,)

2 - size of

3 - Pointer operator (*, &)

4 - Member selection operator.

BIT-WISE OPERATOR:-

There name

→ They work on individual bits

Bitwise AND (&)

- It means into
- means if there are 1,1 then output will be 1 but else it is '0'
- ✓ 1,1 thile 1 neba neheba zero.

Bitwise OR (|)

- It means addition
- 0,0 thile '0' else '1'

Bitwise Exclusive AND (^)

- 0,0 (1,1) thile 1 else '0'
- if same element output will be 0 else 1

Truth table

A	B	$A \& B$	$(A B)$	(A^B)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Left shift

Shift operator are used to move the bit pattern either left or right.

→ Ex $x = 000\ 010\ 110\ 011$
 $x \ll 3 \Rightarrow 010\ 110\ 011\ 000$

Right shift

→ it is used to move the bit pattern towards right

→ Ex $x = 0100\ 1011\ 0101\ \underline{1111}$
 101

$x \gg 4$

→

0000 0100 1011 0101

EVALUATION OF EXPRESSION :-

1- ARITHMETIC EXPRESSION :-

→ An arithmetic expression is a combination of number, (digits) variable and arithmetic operators that the computer evaluates to produce numeric result

→ It is used to perform mathematical calculation.

Ex:- $a=5, b=6, c=7$

$$z = a + b * c / a$$

$$= 5 + 6 * 7 / 5$$

$$= 13.5$$

2- OPERATOR PRECEDENCE :-

→ It means which operation should be get on first

✓ High priority operator - multiplication, division

✓ Low priority operator - Addition, subtraction

Example → $a * b + c$ $a=2, b=3, c=7$

$$\Rightarrow 2 * 3 + 7 = 6 + 7 = 13$$

MATHEMATICAL FUNCTION :-

✓ math.h → It is the standard header file for mathematical function.

Ex:- trigonometry

$$a \cos(x)$$

$$a \sin(x)$$

$$a \tan(x)$$

$$a \tan(x+y)$$

Hyperbolic

$$\cosh(x)$$

$$\sinh(x)$$

Defn:-

CONTROL STRUCTURES

- Control structures in C are the statements that control flow of execution of a program.
- They allow the programmer to decide which instruction to execute, how many times to execute and under what condition.

Types:-

- 1- Sequence control structure
- 2- Selection (Decision Making) control structure
- 3- Looping/iterative control structure

SEQUENCE CONTROL STRUCTURE:-

- The sequence C.S. means statements are executed one after one/another, in the same order as written.

Ex:-

```
int a = 5;  
int b = 10;  
int sum = a + b;  
printf("%d", sum);
```

SELECTION CONTROL STRUCTURE:-

- Used when a program needs to choose between multiple paths based on conditions.

Types:-

- if statement
- if-else
- else-if ladder
- nested if
- switch case
- conditional

1- if statement

Defn:- Executes a block only if the given condition is true.

Syntax:-

```
if (condition)
{
    // statement
}
```

2- if-else statement

Defn:- Executes one block if the condition is true otherwise executes another block (else part).

Syntax:-

```
if (condition)
{
    // statement
}
else
{
    // statement
}
```

if it is true
this will be execute

→ not true then
→ then this part execute

3- else-if ladder :-

Defn:- if there are multiple condition (more than 2) then we use else-if ladder.

→ first if block executes if true if execute and stop if 1st condition is not true then else-if part execute else the else part execute.

Syntax :-

```
if (condition 1)
{
    // statement
}
else if (condition 2)
{
    // statement 2
}
else
{
    // statements
}
```


4. Nested if-else

Nested if-else means an if-else statement within another if-else statement

Syntax:-

```
if (condn 1)
{
    if (condn 2)
    {
        statement
    }
    else
    {
        statement
    }
}
else
{
    if (condn 3)
    {
        statement
    }
    else
    {
        statement
    }
}
```

5. Switch case

- Used when you want to select one option from multiple constant choices

Syntax:-

```
switch (expression)
{
    case 1:
        ...
        break;
    case 2:
        statement
        break;
    case :
        ...
        break;
    default:
        break;
}
```


LOOPING

Defn:- Looping in C refers to executing a block of statements repeatedly until a specific condition becomes false.

- it helps avoid writing the same code again and again.
- it reduces code repetition.
- saves time and memory.
- Makes program shorter and efficient.

} Features of Loop

Types of Loops in C

- 1- While loop
- 2- do while loop
- 3- for loop

1- While loop (Entry-controlled loop)

Defn:- it checks the condition first before executing the body of the loop.

- if the condition is true, only then the statements inside the loop execute.

Syntax

```
while (condition)
{
    // body of loop
}
```

FLOW CHART :-

- condition - true - Execute loop body - check condition Again
- condition - false - Exit from the loop

Example:-

```
int i = 2; // initialization
while (i <= 5) // condition
{
    printf("%d", i);
    i++; // change in value
}
```


2- do-while-loop (Exit-controlled loop)

- A do-while loop executes the body at least once, even if the condition is false.
- The condition is checked after executing the loop body.

Syntax:-

```
do
{
    // statements
} while (condition);
```

Example

```
int i = 1; // initialization
do
{
    printf("%d", i); // execution of body of loop
    i++; // change in value
} while (i <= 5); // condition
```

3- For-Loop

- For loop is used when the number of iteration is known earlier.
- Initialization, condition, and increment/decrement are written in a single line.

Syntax:-

```
for (initialization; condition; change in value)
{
    // statements
}
```

Example :-

```
for (int i = 1; i <= 5; i++)
{
    printf("%d", i);
}
```

Annotations for the example code:
- **initialization** points to `int i = 1;`
- **condition** points to `i <= 5;`
- **change in value** points to `i++`

NESTING OF LOOP

- if a loop present inside another loop it is called as Nesting of loop.

syntax:-

```
for (initialization; condition; change in value)
{
    for (initialization; condition; change in value)
    {
        statement
    }
}
```

JUMPS IN LOOP :-

We use "break" statement to jumping out of loop

syntax

while loop

```
initialization
while (condition)
{
    statement
}
if (condition)
{
    break;
}
statement
}
```

do-while

```
initialization
do
{
    statement
} while (condition);
if (condition)
{
    break;
} while (condition);
```

FOR-LOOP

```
for (initialization; condition; incre/decre)
{
    for (initialization; condition; incre/decre)
    {
        statement
        if (condition)
        {
            break;
        }
        statement
    }
}
```


SKIPPING PART OF A LOOP

Defn:-

syntax:-

```
initialization
while (condition)
{
    if (condition)
    {
        continue;
    }
    statement;
    i++;
}
```

Example:-

```
#include <stdio.h>
void main ()
{
    int i = 1;
    while (i <= 20)
    {
        if (i > 5 && i <= 10)
        {
            continue;
        }
        printf(" %d ", i);
        i++;
    }
}
```

output

1, 2, 3, 4, 5, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20

ARRAY

Defn:- An Array is a collection of similar datatypes stored in contiguous memory location and referred by unique name

Syntax:-

datatype array-name [size];

Ex - int number [5];

CHARACTERISTICS OF AN ARRAY

- All elements are of same size
- Elements are stored in contiguous memory blocks.
- Element are accessed through index number.

Types of Array:-

- 1- one - dimensional (1D)
- 2- Two - dimensional (2D)
- 3- Multi Dimensional (3D, 4D)

ONE - DIMENSIONAL (1D)

Two Dimensional Array (2D-Array)

Syntax:-

Data type array-name [row] [column];

Ex:-

```
int arr[3][3];
printf("Enter the elements");
for (i=1; i<=3; i++)
{
    for (j=1; j<=3; j++)
    {
        scanf("%d", &arr[i][j]);
    }
}
for (i=1; i<=3; i++)
{
    for (j=1; j<=3; j++)
    {
        printf("%d", arr[i][j]);
    }
    printf("\n");
}
```

CREATING CHARACTER ARRAY:-

Syntax:-

char name [size];

A character array is known as string.

DECLARING AND INITIALISING CHARACTER ARRAY:-

Syntax:-

char variable-name [size]

Ex:-

char city [20]

char name [30]

Initialising

char city [20] = "Bangalore";

char city [20] = {'B', 'a', 'n', 'g', 'a', 'l', 'o', 'r', 'e'};

READING A STRING FROM USER:-

char city [20];

scanf("%s", city);

Writing a string - printf("%s", city);

STRING HANDLING FUNCTION :-

(string.h) → It is a library file
- it is used for handling the string function

Some string handling function are

1. str cat()
2. str cmp()
3. str copy()
4. str len()

1. STRING CONCATINATION (str cat)

- str cat() - function join two string together.

Syntax.

str cat (s1, s2);

→ s2 is append to s1

Ex:-

s1 = {"Manthan"}

s2 = {"_sahu"}

s3 = str cat (s1, s2);

printf ("%s", s3); // Manthan_sahu

NOTE. This function take only 2 argument

2 If there are 3 strings like this -

→ str cat (str cat (s3, s1), s2); → This will be the syntax

→ first s1 append to s3 and then s2 append to the first appended string.

2. str cmp()

→ str cmp() function compare 2 string identified by argument and has a value '0' if they are equal.

→ if they are not equal it has a numeric difference between the first non-matching character (According to their ASCII value)

Ex:- s1 = "abc"

s2 = "Abc"

s3 = str cmp (s1, s2);

printf (s3); // 97 - 65 = 32

strncpy()

strncpy() - function copy one string into another string.

Ex:-

strncpy(s1, s2)

Here argument of s2 will be copied to s1

Ex:-
char city[20]
s1 = strncpy(city, "Delhi");
printf("%s", s1);

output

Delhi

strlen()

strlen() - function counts the number of character in the string and return the va integer value

Syntax:-

n = strlen(s)

→ variable of string

Ex:-

str1[] = "Nanthan - Sahu";

n = strlen(str1);

printf("%d", n); // 12

FUNCTION

Defn:- Function is a self contained block of code that is designed to perform a specific task.

→ Generally function is of two types:-

1- Build in function

2- User define function.

1- Build-in Function

These are function that are already provided by 'C'.

Ex-

printf(), scanf(), strcpy(), strcmp(), getch()

2- User-define-function

- User-define function means user declare own function as per requirement.

- Generally User-Define function have

1- function Declaration

2- function Defination

3- function call

1. Function Declaration

Syntax:-

Return-type function-name (arguments);

Ex:-

int sum (int, int);

Return
type

void sum (int, int);

2. Function Defination

In this part the body of the function or executable part has been written.

syntax

return-type

function-name (arguments)

{

statements

}

3. Function Call

Use to call the function

syntax

function-name (arguments);

There are 4 types of User-Defined function:-

- 1- With return type with Argument - `int sum (int, int);`
- 2- With return type without Argument - `int sum ();`
- 3- Without return type with Argument `void sum (int, int);`
- 4- Without return type without Argument - `void sum ();`

ACTUAL ARGUMENT :-

The argument that are passed at the time of function call is known as Actual Argument.

FORMAL ARGUMENT :-

The argument that are passed at the time of function definition is known as formal Argument.

Called by VALUE :-

```
void change (int x); // function declaration
```

```
void main ()
```

```
{
```

```
    int a;
```

```
    printf("enter a Number:");
```

```
    scanf ("%d", &a);
```

```
    printf ("a = %d", a);
```

```
    change (a); // function call
```

```
void change (int x) // function definition
```

```
{
```

```
    x = x * x
```

```
    printf ("x = %d", x);
```

```
}
```

```
}
```


Call-By-Reference

```
void change (int *x); // function declaration
```

```
void main ()
```

```
{
```

```
    int a;
```

```
    printf ("Enter a Number :");
```

```
    scanf ("%d", &a);
```

```
    printf ("a = %d", a);
```

```
    change (&a); // function call
```

```
    printf ("a = %d", a);
```

```
}
```

```
void change (int *x)
```

```
{
```

```
    x = *x + *x;
```

```
    printf ("x = %d", *x);
```

```
}
```

} function definition

N.B $x = \text{Address}$ $*x = \text{value at address};$

Any changes made to the address will reflect in the original value.

NESTING OF FUNCTION :-

When there are more than one function we can call a function inside another function.

```
function1();
```

```
function2();
```

```
void main()
```

```
{
```

```
    void function1();
```

```
    void function2();
```

```
    void function3();
```

```
    printf ("inside function");
```

```
function1()
```

```
{
```

```
    - - - -
```

```
}
```

```
function2()
```

```
{
```

```
    - - - -
```

```
    function1();
```

```
}
```


Recursion

PASSING ARRAY TO A FUNCTION :

syntax:-

return-type function (array, size);
or
 return-type function (arr[], size);

SCOPE OF VARIABLE AND life time of 'VARIABLE'.

STORAGE CLASS	SCOPE	WHERE DECLARE	visibilty	life
none		Before all fun in a file	Entire file	Entire program
Auto	Local	Inside a fun or block	only in the block / fun	untill the end of block
Extern	Global	Before fun in a file	Entire file	Global
Register	Local	inside a fun	only in the fun / block	untill end- of the fun
static	Global	inside a fun	only in the fun	Global

POINTER

Defn:- Pointer is a variable that is used to store the address of another variable.

Pointer Declaration :-

Syntax

datatype *pointer - name;
Ex:- int *point; *p = &x // error
char *p;

Initialising a pointer:-

syntax:- pointer variable = &variable

Ex:- int *p;
p = &a; → pointing address of a.

Accessing the value through its pointer:-

```
let int a = 25
printf ("%d", a) // output 25
printf ("%d", p) // Address of a
printf ("%d", *p); // display value at address.
```

Pointer Expression :-

- Pointer Expression is same as Arithmetic Expression.

Ex:-
int a, b, *p, *q
int c = a * b // Arithmetic Expression
int d = *a * *b // pointer Expression
 ↘ multiplication.

*p++ - value at Address is increase

p++ - Address is increase

RULES OF POINTER VARIABLE OPERATION

- A pointer can store the address of a variable of the same type.

```
int a = 10;
```

```
int *p = &a; // valid
```

```
int *p;
```

```
float x = 5.1;
```

```
p = &x; // invalid
```

store the address of

- A pointer of one type can't store the address of another type.

- Only two arithmetic operations are allowed on pointer.

1- Addition

2- Subtraction.

- Pointers can be added or subtracted only with integers, but cannot be added to another pointer.

```
p = p + 2; // valid
```

```
p = p - 2; // valid
```

```
p = p + q; // invalid
```

- We cannot add two pointers but you can subtract them if they point to the same array.

```
int diff = p - q; // valid.
```

- pointers can be compared only when both belong to the same memory block.

- Dereferencing means accessing the value stored at the pointer's address.

Do NOT dereference:

- uninitialized pointer

- Null pointer

- freed memory pointer

Void Pointer

- A void pointer can store address of any data type.
- But it cannot be directly dereferenced.

Pointer type must match the data type.

- The array name behaves like a pointer to the first element.

```
int arr[5];
int *p = arr;
```

- Pointer increment depends on Data type
- ✓ A pointer that is not initialized is called a wild pointer
- When memory is freed the pointer becomes dangling.

Pointer AND Array :-

X =

5	8	9	12	15
---	---	---	----	----

0 1 2 3 4 → index

```
Ans. #include <stdio.h>
void main ()
{
    int *p, sum;
    int x[5] = {5, 8, 9, 12, 15};
    i = 0; p = x;
    printf("Element - Value Address");
    while (i < 5)
    {
        printf("%x [%d] %d %u \n", i, *p, p);
        sum = sum + *p;
        i++; p++;
    }
    printf("\n sum = %d \n", sum);
    printf("\n &x[0] = %u \n", &x[0]);
    printf("\n sum = %u \n", AP);
}
```


Defn:-

STRUCTURE is a user defined data type in C that allows you to combine variables of different data types under a single name.

In simple words:-

Structure - group of different data types stored together.

Syntax

```
struct struct-name {
```

```
    data-type member 1;
```

```
    data-type member 2;
```

```
}
```

Ex:- struct student

```
{
```

```
    int roll;
```

```
    float marks;
```

```
    char name[20];
```

```
}
```

Structure Member

STRUCTURE VARIABLE :-

Syntax:-

```
struct structure-name variable;
```

Ex:- struct student s1, s2, s3;

↳ structure variables

ACCESSING STRUCTURE MEMBER :-

Syntax:-

```
structure variable . structure-member;
```

We can access structure member by '.' - dot membership operator.

↳ We can also access structure member through scanf() - function.

Ex. s1.roll
s1.name

Ex:-

```
#include <stdio.h>
struct student
{
    char Name [10];
    int Roll;
    float marks;
```

}; *→ We can also declare structure variable here*

```
void main ()
```

```
{
    struct student s1, s2;
    s1. Name = "Manthan";
    s1. Roll = 18;
    s1. Marks = 100;
    s2. Name = "Khushi";
```

```
printf ("Student 1");
printf ("Name = %s", s1. Name);
printf ("Roll Number = %d", s1. Roll);
```

```
}
```